GUÍA DEL ALUMNO

El estado cambia. La pantalla se entera.

Una tienda de cursos construida paso a paso: **ref, reactive, computed, watch, directivas y lo nuevo de Vue 3.5 y 3.6.**

Estructura y reactividad en Vue

Vue 3.5 · Composition API · TypeScript

Oscar Alvarado

Frontend Tech Lead

Universidad San Martín de Porres

Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

Una tienda de cursos, en seis pasos

En clase construimos **una sola app**: una tienda de cursos con catálogo, carrito y cupón. Vive en un único archivo, `Tienda.vue`, y cada ejemplo le agrega una pieza. Antes de cada ejemplo está la **teoría** de lo que se usa: qué es cada herramienta, para qué sirve y un diagrama de cómo funciona por dentro, el mismo de las slides. Después, los pasos del ejemplo en orden, diciendo **dónde** va cada cosa. El código completo está al final de cada bloque.

1
Reactividad
`ejemplos 1–3 · ref · reactive · computed · watch`

2
Renderizado y directivas
`ejemplos 4–5 · v-if · v-for · modificadores`

3
Vue 3.5 y 3.6
`ejemplo 6 · props reactivos · alien-signals`

Los seis pasos de la Tienda

- ✓ 1. Carrito y alumno · ref, reactive
- ✓ 2. Totales · computed
- ✓ 3. Guardar y cupón · watch
- ✓ 4. Catálogo con estados · v-if, v-for
- ✓ 5. Formularios · modificadores, v-focus
- ✓ 6. ItemCarrito · props reactivos

La Tienda por dentro

Un script y tres secciones en el template, marcadas con comentarios: **Alumno** (nombre y email), **Catálogo** (los cursos) y **Carrito** (líneas, cupón y totales). Al final, cada línea del carrito será su propio componente: `ItemCarrito.vue`.

Lo que necesitas instalado

- **Node.js 22.18 o superior** (comprueba con `node -v`).
- **VS Code** con la extensión **Vue (Official)**.
- El navegador con las **DevTools** abiertas (F12): vamos a usar las pestañas Console, Elements y Application.

ARRANQUE · UN PROYECTO LIMPIO 5 MIN

Un comando y dos archivos de partida

terminal

```
npm create vue@latest reactividad -- --ts --bare
cd reactividad
npm install
npm run dev
```

`--ts` activa TypeScript y `--bare` crea el proyecto **sin el código de ejemplo** de Vue, así no hay nada que borrar.

Crea `src/cursos.ts` con los datos (cópialo, no hace falta escribirlo):

src/cursos.ts

```ts
export interface Curso {
  id: number
  titulo: string
  precio: number
  categoria: 'frontend' | 'backend' | 'datos'
}

export const CURSOS: Curso[] = [
  { id: 1, titulo: 'Vue desde cero', precio: 120, categoria: 'frontend' },
  { id: 2, titulo: 'TypeScript práctico', precio: 90, categoria: 'frontend' },
  { id: 3, titulo: 'Node y Express', precio: 110, categoria: 'backend' },
  { id: 4, titulo: 'Bases de datos SQL', precio: 100, categoria: 'datos' },
  { id: 5, titulo: 'APIs con NestJS', precio: 150, categoria: 'backend' },
  { id: 6, titulo: 'Python para datos', precio: 130, categoria: 'datos' },
]
```

Y reemplaza `src/App.vue`. Toda la clase trabajamos en `src/components/Tienda.vue`, que creamos en el ejemplo 1.

src/App.vue

```vue
<script setup lang="ts">
// La Tienda de cursos: la construimos paso a paso en clase
import Tienda from './components/Tienda.vue'
</script>

<template>
  <main>
    <Tienda />
  </main>
</template>

<style>
body {
  font-family: system-ui, sans-serif;
  line-height: 1.5;
  max-width: 720px;
  margin: 2rem auto;
  padding: 0 1rem;
}
input,
select,
textarea,
button {
  font: inherit;
  padding: 0.25rem 0.5rem;
}
section {
  margin-block: 1.5rem;
}
</style>
```

BLOQUE 1 · REACTIVIDAD 32 MIN

Dos guardan, una calcula, una reacciona

HERRAMIENTA	PARA QUÉ	EN LA TIENDA
`ref`	Guardar un valor: número, texto, array…	`carrito`, `cupon`
`reactive`	Guardar un objeto cuyos campos van juntos	`cliente`
`computed`	Calcular un valor a partir de otros	`total`, `visibles`
`watch` · `watchEffect`	Hacer algo hacia afuera cuando algo cambia	localStorage, título

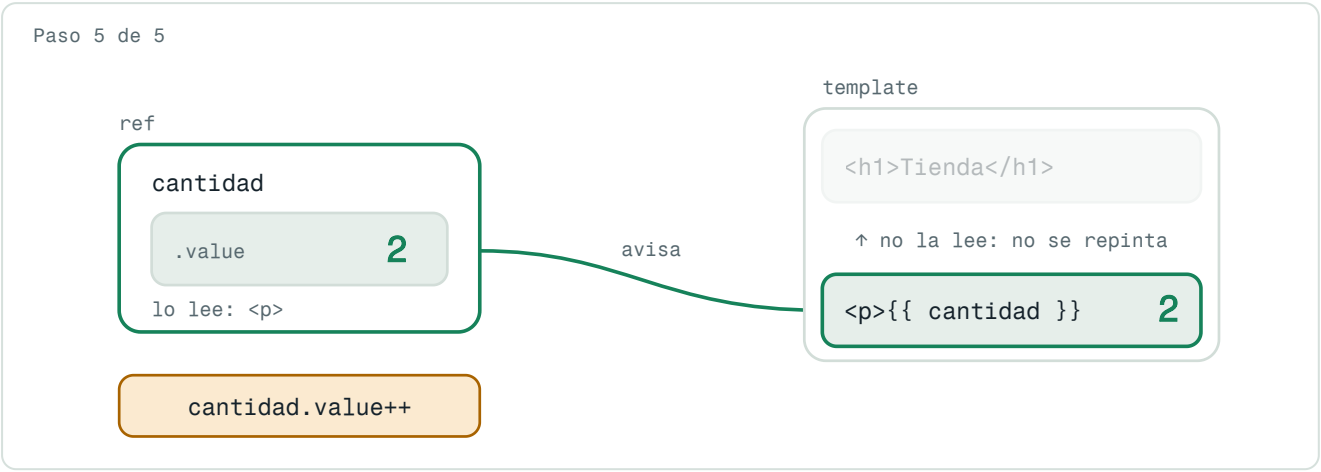
CONCEPTO `ref`

QUÉ ES

Una caja que guarda **un valor** y avisa cuando cambia.

PARA QUÉ SE USA

Cualquier dato que, al cambiar, deba verse en pantalla: un número, un texto, un array.



Cómo funciona, paso a paso

1. `ref(1)` crea una caja. El dato vive en `.value`.
2. El template lee `cantidad`: Vue anota que ese texto **depende** de la caja.
3. En el script escribes `cantidad.value++`.
4. La caja avisa solo a quien la leyó.
5. Se repinta ese texto y nada más. El título no la lee: ni se entera.

REGLA

En el script, `.value`. En el template no: Vue lo quita por ti.

CONCEPTO

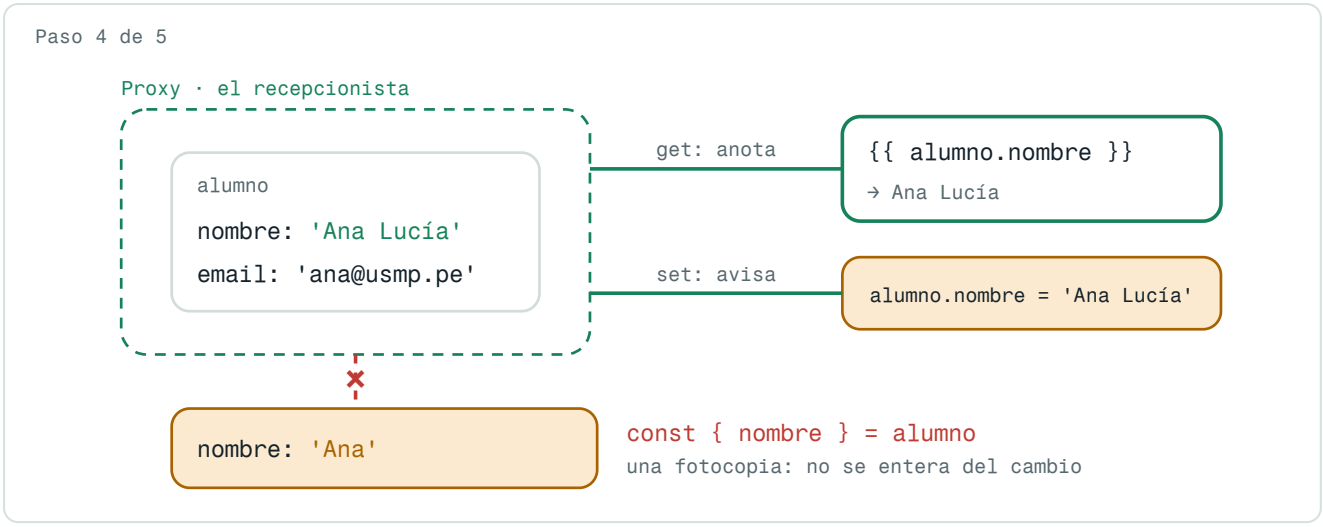
reactive

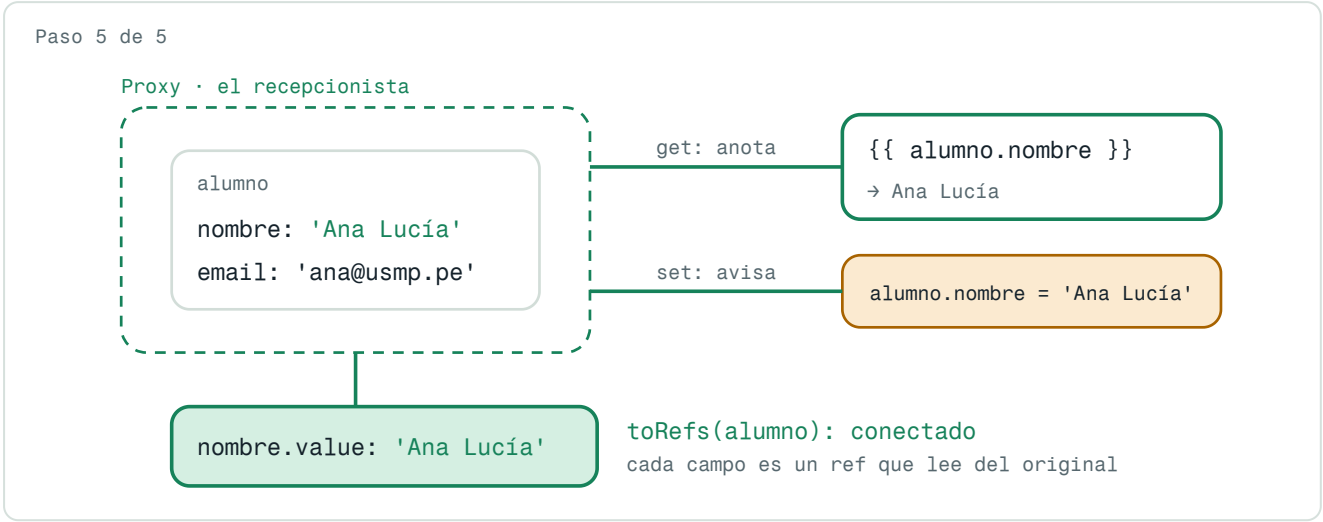
QUÉ ES

Vuelve reactivo **un objeto entero**, sin `.value`.

PARA QUÉ SE USA

Datos que van juntos y se usan completos, como los campos de un formulario.





Cómo funciona, paso a paso

1. `reactive` envuelve el objeto en un **proxy**: un recepcionista delante de los datos.
2. Cada lectura pasa por el recepcionista: anota quién lee `alumno.nombre`.
3. Cada escritura también: avisa a todos los que leyeron ese campo.
4. `const { nombre } = alumno` saca una **fotocopia**: el valor, sin recepcionista. La copia no se entera.
5. `toRefs(alumno)` entrega cables, no copias: cada campo sigue conectado.

REGLA

`ref` por defecto. `reactive` solo para un objeto que siempre usas completo, y **nunca lo desestructures** sin `toRefs`.

Ejemplo 1 de 6 · Carrito y alumno

· 7 min

Creamos la Tienda: un catálogo, un carrito que suma cantidades y los datos del alumno.

1. **Archivo nuevo** — `src/components/Tienda.vue` con un `<h1>` y la sección **Catálogo**: un `v-for` sobre `CURSOS`.
2. **Script** — la interfaz `Linea` (curso + cantidad), `carrito = ref<Linea[]>([])` y `agregar(curso)`: si ya está, `cantidad++`; si no, `push`. **Template** — botón "Agregar" en cada curso y la sección **Carrito**.
3. **Script** — `cliente = reactive({ nombre: '', email: '' })`. **Template** — sección **Alumno** con dos inputs `v-model`, entre el título y el catálogo.
4. **Script** — `const { nombre } = cliente` y **Template** — `Pedido de {{ nombre }}` debajo del título: escribe en el nombre y **no cambia**.
5. Cámbialo por `const { nombre } = toRefs(cliente)`: ahora sí se actualiza.

Fíjate también en `linea.cantidad++`: cambiamos algo **dentro** del array y la pantalla se entera, porque `ref` vigila también lo de dentro.

CONCEPTO computed

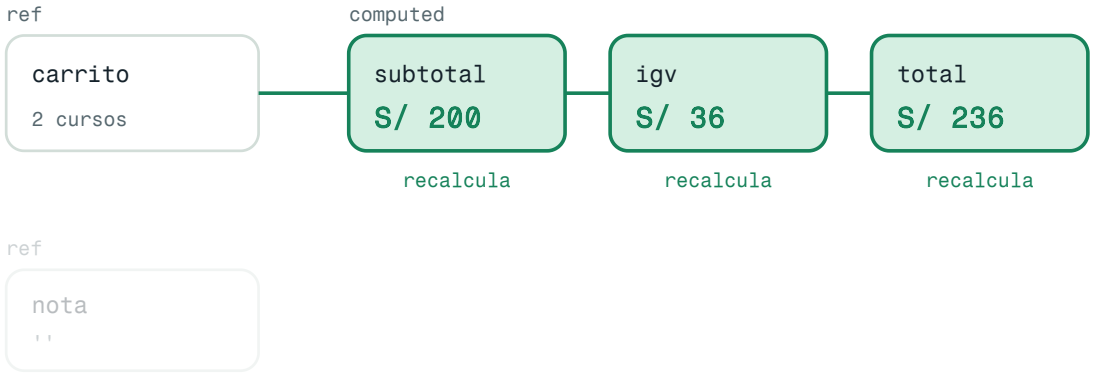
QUÉ ES

Un valor que **se calcula** a partir de otros, como una fórmula de Excel.

PARA QUÉ SE USA

Totales, filtros, conteos: todo lo que se puede derivar de datos que ya tienes.

Paso 4 de 5





Cómo funciona, paso a paso

1. Un `computed` lee otros datos y devuelve un valor. Uno puede depender de otro.
2. La primera vez que se lee, calcula y **guarda** el resultado: la caché.
3. Cambia el carrito: toda la cadena que depende de él queda marcada como **sucia**.
4. Cuando alguien vuelve a leer, recalcula subtotal → IGV → total.
5. Escribes en «Nota»: nadie de la cadena la lee, así que no recalcula nada. Una función normal sí se ejecutaría otra vez.

REGLA

Un `computed` calcula y devuelve. **Sin efectos secundarios**: nada de guardar, llamar a una API ni cambiar otras variables.

Ejemplo 2 de 6 · Totales

· 6 min

La Tienda calcula sola cuántos cursos llevas, el subtotal, el IGV y el total.

1. **Script · debajo de agregar** — `cantidadTotal` con `computed` y `reduce`. **Template** — el saludo pasa a `Pedido de {{ nombre }} · {{ cantidadTotal }} cursos`.
2. **Script** — `subtotal`, `igv` y `total`, encadenados. **Template · Carrito, debajo de la lista** — los tres importes.
3. Demo de la caché: un `subtotalMetodo()` temporal con `console.log`. Al escribir en el **nombre** del alumno solo se ejecuta la función, nunca el `computed`.
4. Borra la función de prueba y los `console.log`.

CONCEPTO

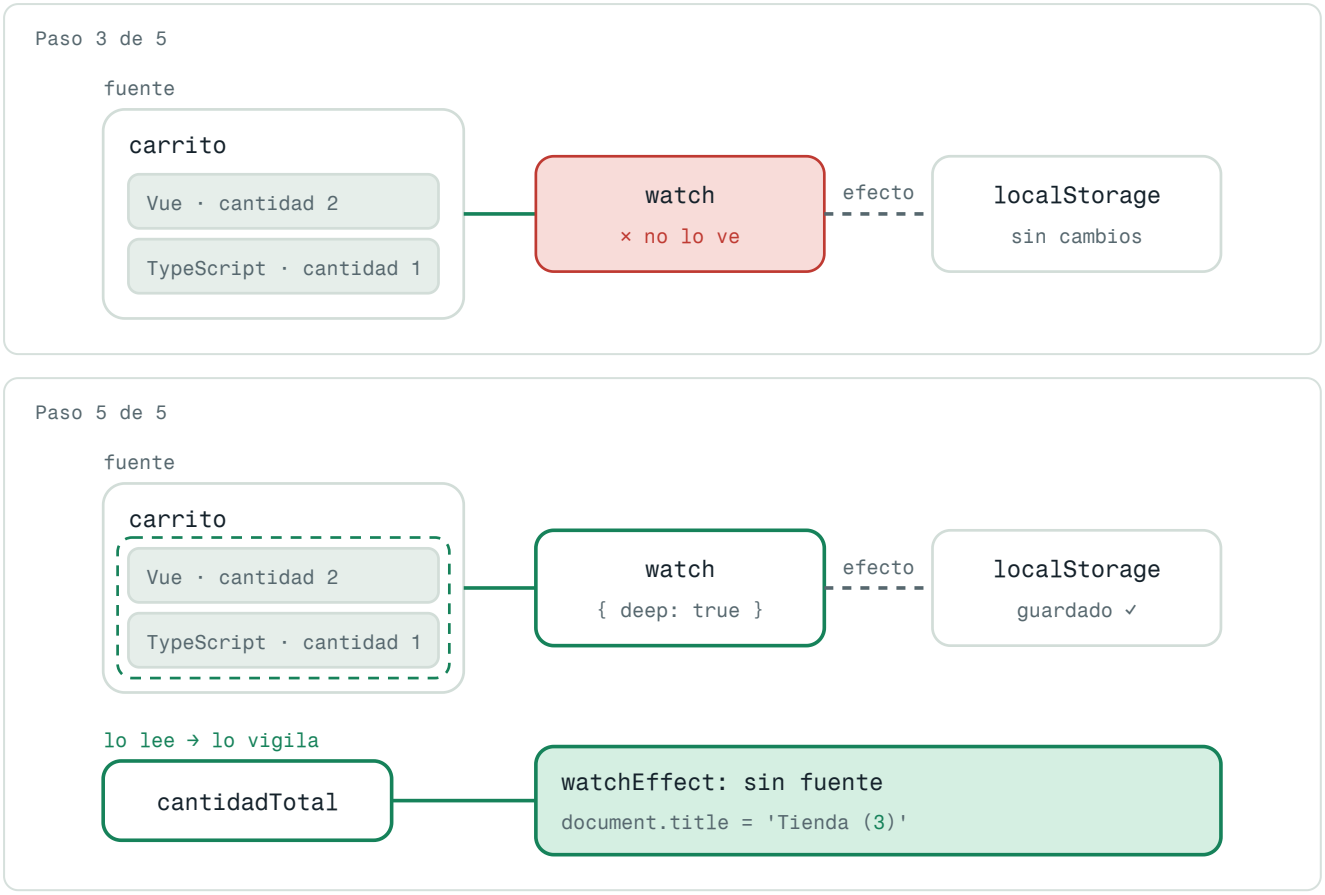
watch · watchEffect

QUÉ ES

Ejecuta un **efecto** cuando algo cambia: algo que el código hace hacia afuera.

PARA QUÉ SE USA

Guardar en el navegador, llamar a una API, cambiar el título de la pestaña.



Cómo funciona, paso a paso

1. `watch(carrito, guardar)`: tú le dices **qué vigilar** y qué hacer cuando cambie.
2. Se reemplaza el carrito → se ejecuta el efecto: guardar en localStorage.
3. Sin `deep`, solo ve que cambies la caja entera. Una cantidad **dentro** no lo dispara.
4. Con `{ deep: true }` mira dentro: cualquier cambio interno dispara el efecto.
5. `watchEffect` no recibe fuente: se ejecuta de inmediato y vigila **todo lo que lee**.

REGLA

Si se puede calcular, `computed`. Si hay que **hacer algo hacia afuera**, `watch`.

Ejemplo 3 de 6 · Guardar el carrito y el cupón

· 7 min

El carrito sobrevive a recargar la página y un cupón aplica un descuento al total.

1. **Script · debajo de agregar** — `vaciar()`. **Template** — botón "Vaciar" en el título del carrito.

2. **Script** — `watch(carrito, …)` que guarda en `localStorage`. Sin `{ deep: true }`, ni "Agregar" ni el `cantidad++` se guardan; "Vaciar" sí. Con `deep`, todo.
3. **Script** — al crear `carrito`, lee lo guardado con `JSON.parse`. Recarga: el carrito sigue ahí.
4. **Script** — `cupon`, `mensaje` y `descuento`; un `watch(cupon, (nuevo, anterior) => …)` que simula la consulta con `setTimeout` y pone `descuento`. `total` pasa a restar el descuento. **Template · Carrito** — el input del cupón y el mensaje.
5. **Script** — `watchEffect` que pone `cantidadTotal` en el título de la pestaña.

	WATCH	WATCHEFFECT
¿Qué vigila?	Lo que tú le dices	Todo lo reactivo que lee
¿Valor anterior?	Sí	No
¿Se ejecuta al crearse?	No, espera al primer cambio	Sí, de inmediato

OJO · EL REPARTO WATCH / COMPUTED

El `watch` del cupón hace el efecto (consultar) y guarda el resultado en `descuento`; el `total` se recalcula solo porque depende de `descuento`. El mensaje no puede ser un `computed` porque hay una **espera**.

Así queda la Tienda al terminar el bloque 1 (**checkpoint 3**):

src/components/Tienda.vue · checkpoint 3

```
<script setup lang="ts">
import { ref, reactive, toRefs, computed, watch, watchEffect } from 'vue'
import { CURSOS, type Curso } from '@/cursos'

interface Linea {
  curso: Curso
  cantidad: number
}

const CLAVE = 'tienda-carrito'
const guardado = localStorage.getItem(CLAVE)
const carrito = ref<Linea[]>(guardado ? JSON.parse(guardado) : [])

function agregar(curso: Curso) {
  const linea = carrito.value.find((l) => l.curso.id === curso.id)
  if (linea) linea.cantidad++
  else carrito.value.push({ curso, cantidad: 1 })
}
function vaciar() {
  carrito.value = []
}

const cupon = ref('')
const mensaje = ref('')
const descuento = ref(0)

const cantidadTotal = computed(() => carrito.value.reduce((suma, l) => suma + l.cantidad, 0))
const subtotal = computed(() =>
  carrito.value.reduce((suma, l) => suma + l.curso.precio * l.cantidad, 0),
```

```
)
const igv = computed(() => subtotal.value * 0.18)
const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))

watch(
  carrito,
  (nuevo) => {
    localStorage.setItem(CLAVE, JSON.stringify(nuevo))
  },
  { deep: true },
)

watch(cupon, (nuevo, anterior) => {
  console.log(`cupón: "${anterior}" → "${nuevo}"`)
  mensaje.value = 'Validando…'
  setTimeout(() => {
    const valido = nuevo === 'VUE2026'
    descuento.value = valido ? 0.1 : 0
    mensaje.value = valido ? '✅ 10% de descuento' : '❌ Cupón no válido'
  }, 800)
})

watchEffect(() => {
  document.title = `Tienda (${cantidadTotal.value})`
})

const cliente = reactive({ nombre: '', email: '' })
const { nombre } = toRefs(cliente)
</script>

<template>
  <h1>Tienda de cursos</h1>
  <p>Pedido de {{ nombre }} · {{ cantidadTotal }} cursos</p>

  <!-- Alumno -->
  <section>
    <h2>Alumno</h2>
    <input v-model="cliente.nombre" placeholder="Nombre" />
    <input v-model="cliente.email" placeholder="Email" />
  </section>

  <!-- Catálogo -->
  <section>
    <h2>Catálogo</h2>
    <ul>
      <li v-for="curso in CURSOS" :key="curso.id">
        {{ curso.titulo }} · S/ {{ curso.precio }}
        <button @click="agregar(curso)">Agregar</button>
      </li>
    </ul>
  </section>

  <!-- Carrito -->
  <section>
    <h2>Carrito <button @click="vaciar">Vaciar</button></h2>
    <ul>
      <li v-for="linea in carrito" :key="linea.curso.id">
        {{ linea.curso.titulo }} × {{ linea.cantidad }}
      </li>
    </ul>
    <input v-model="cupon" placeholder="Cupón de descuento" />
    <p>{{ mensaje }}</p>
    <p>Subtotal: S/ {{ subtotal.toFixed(2) }}</p>
```

```
    <p>IGV (18%): S/ {{ igv.toFixed(2) }}</p>
    <p><strong>Total: S/ {{ total.toFixed(2) }}</strong></p>
  </section>
</template>
```

BLOQUE 2 · RENDERIZADO Y DIRECTIVAS 24 MIN

Toda lista tiene tres estados: cargando, vacía y con datos

DIRECTIVA	QUÉ HACE	ÚSALA SI…
`v-if · v-else-if · v-else`	Crea o destruye el elemento	cambia poco
`v-show`	El elemento siempre está; lo oculta con CSS	cambia seguido
`v-for + :key`	Una copia por elemento	listas
`<template>`	Bolsa invisible: agrupa sin agregar etiquetas	v-if/v-for a varios

CONCEPTO `v-if · v-show`

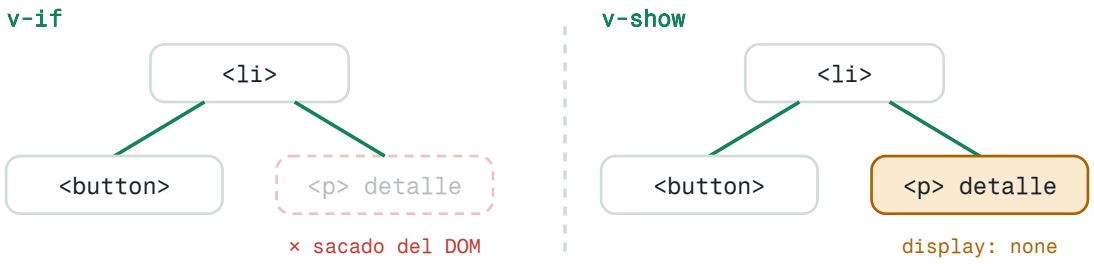
QUÉ ES

Dos formas de mostrar u ocultar: `v-if` crea y destruye, `v-show` solo oculta.

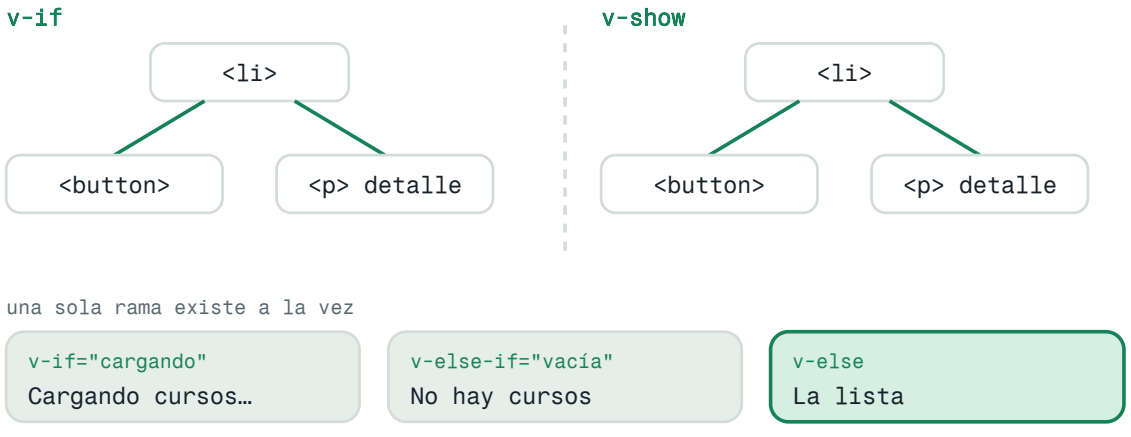
PARA QUÉ SE USA

Estados de carga, mensajes vacíos, detalles que se abren y cierran.

Paso 2 de 4



Paso 4 de 4



Cómo funciona, paso a paso

1. Mismo HTML, dos estrategias: a la izquierda `v-if`, a la derecha `v-show`.
2. Al ocultar: `v-if` **saca** el nodo del DOM; `v-show` lo deja con `display: none`.
3. Al volver: `v-if` lo **crea de nuevo** desde cero; `v-show` solo quita el estilo.

4 Con `v-if` / `v-else-if` / `v-else` resuelves los tres estados de una lista.

REGLA

¿Cambia poco? `v-if`. ¿Se abre y cierra seguido? `v-show`. Y toda lista tiene **tres estados**: cargando, vacía y con datos.

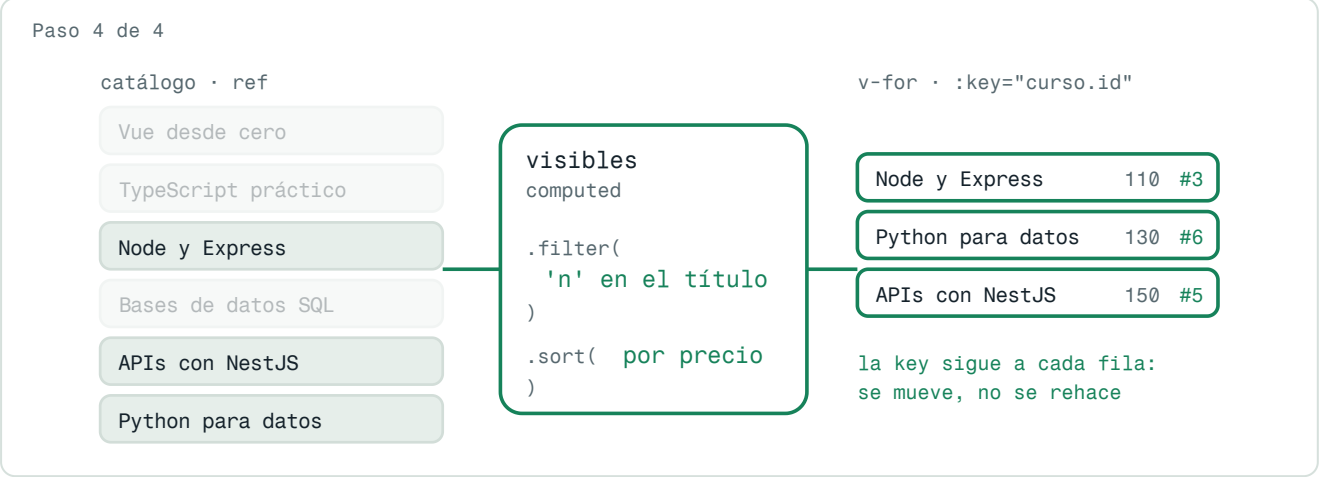
CONCEPTO v-for · :key

QUÉ ES

Dibuja **una copia del elemento por cada dato** de una lista.

PARA QUÉ SE USA

Catálogos, carritos, tablas: cualquier lista que venga de tus datos.



Cómo funciona, paso a paso

1 La fuente es el catálogo completo: seis cursos en un `ref`.

2 Un `computed` lo filtra por lo que escribes en el buscador…

3 …y lo ordena. El template solo recorre el resultado con `v-for`.

4 Cambias el orden: cada fila tiene su `:key` (su DNI), así que Vue la **mueve**; no la rehace.

REGLA

Filtra y ordena en un `computed`, no con `v-if` dentro del `v-for`. Y `:key` siempre es un **id único**, nunca el índice.

Ejemplo 4 de 6 · El catálogo con estados

· 9 min

El catálogo pasa de lista fija a uno real. Todo pasa en el script y en la sección **Catálogo**; el carrito no se toca.

1 **Script · encima de cliente** — `cargando`, `cursos` y un `setTimeout` que simula la API. **Template** — `v-if="cargando"` y la lista con `v-else`, recorriendo `cursos`.

2. **Script** — `busqueda` y un `computed` `visibles` con `filter`. **Template** — el buscador; el `v-for` recorre `visibles`.
3. **Template** — el estado vacío con `v-else-if="visibles.length === 0"`.
4. **Script** — `orden` y `.sort()` después del `.filter()`. **Template** — un `<select>`.
5. **Template** — envuelve el contador y la lista en `<template v-else>`.
6. **Script** — `abierto` y `alternar(id)`. **Template** — el título de cada curso es un botón y el detalle se muestra con `v-show`.

OJO · NO FILTRES CON V-IF DENTRO DEL V-FOR

En el mismo elemento, el `v-if` se evalúa **antes** que el `v-for` y no ve la variable del bucle. Filtra en un `computed`. Y `sort` modifica el array: aquí es seguro porque `filter` ya devolvió uno **nuevo**.

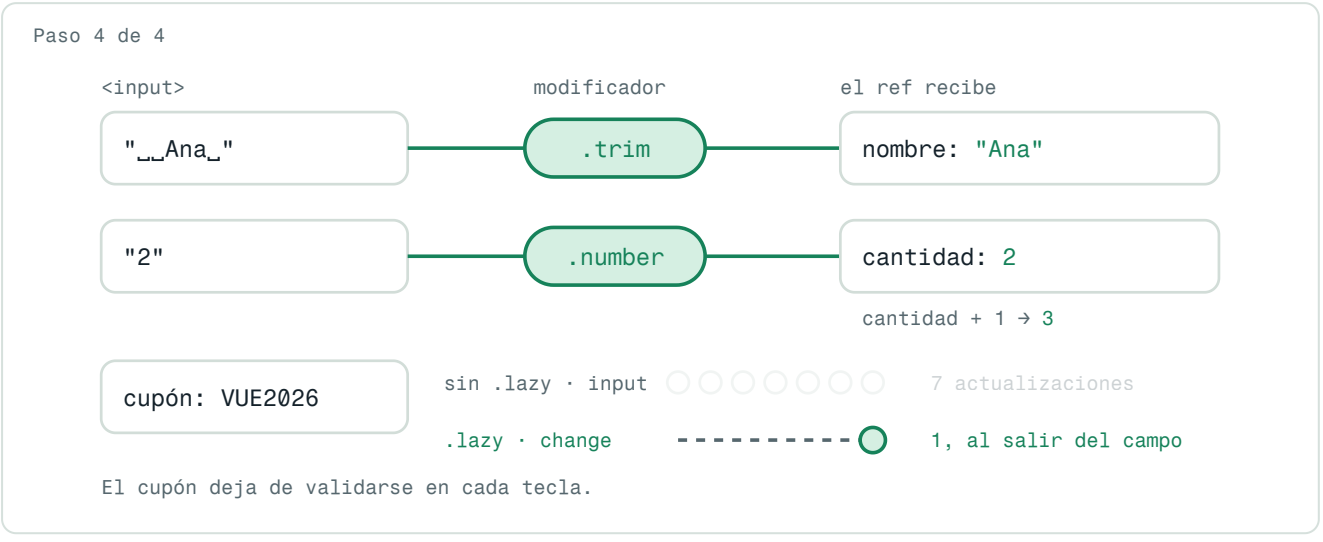
CONCEPTO .trim · .number · .lazy

QUÉ ES

Un ajuste que se pega a `v-model` **con un punto**: filtra el dato antes de guardarlo.

PARA QUÉ SE USA

Limpiar lo que escribe el usuario, convertirlo a número o decidir cuándo se guarda.



Cómo funciona, paso a paso

1. Sin modificadores, `v-model` guarda el texto **tal cual**: espacios incluidos, y siempre como texto.
2. `.trim` quita los espacios de los extremos antes de guardar.
3. `.number` convierte el texto en número: `cantidad + 1` vuelve a sumar.
4. `.lazy` cambia el momento: guarda al salir del campo (`change`), no en cada tecla (`input`).

REGLA

Todo lo que sale de un input es **texto**: `"2" + 1` da `"21"`.

CONCEPTO **v-focus**

QUÉ ES

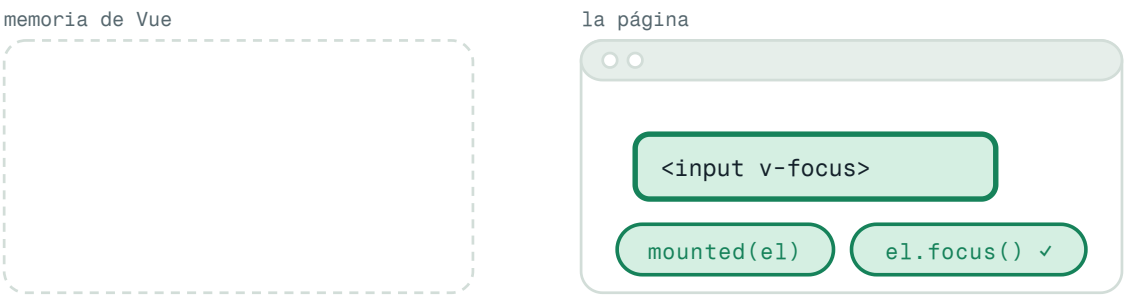
Una **directiva propia**: un objeto con hooks que Vue llama en momentos del elemento.

PARA QUÉ SE USA

Un comportamiento reutilizable sobre el DOM: enfocar, medir, detectar clics afuera.

Paso 4 de 4

```
const vFocus = { mounted: (el) => el.focus() }
```



Cómo funciona, paso a paso

1. La directiva es un objeto con **hooks**: funciones que Vue llama en momentos del elemento.
2. Vue crea el `<input>` en memoria. Todavía no está en la página: no se puede enfocar.
3. Lo inserta en la página y llama a `mounted(el)` con el elemento real.
4. Ahí hacemos `el.focus()`. Y funciona aunque el input aparezca después, con un `v-if`.

REGLA

`vFocus` en el script se usa como `v-focus` en el template. A diferencia de `autofocus`, funciona aunque el elemento **aparezca después**.

Ejemplo 5 de 6 · Formularios: modificadores y v-focus

· 6 min

Cambios pequeños en el carrito, el alumno y el script.

1. **Template · Carrito, dentro de cada línea** — un input `v-model="linea.cantidad"`. Cambia una cantidad a 3: arriba aparece algo como **"031 cursos"**. Con `v-model.number` se arregla.
2. **Template · Alumno** — `v-model.trim` en el nombre: los espacios de los extremos desaparecen.
3. **Template · Carrito** — `v-model.lazy` en el cupón: se valida al salir del campo, no en cada tecla.
4. **Script** — la directiva `vFocus` con el hook `mounted`. **Template · Alumno** — `v-focus` en el nombre.

Así queda la Tienda al terminar el bloque 2 (**checkpoint 5**):

src/components/Tienda.vue · checkpoint 5

```
<script setup lang="ts">
import { ref, reactive, toRefs, computed, watch, watchEffect, type Directive } from 'vue'
```

```
import { CURSOS, type Curso } from '@/cursos'

interface Linea {
  curso: Curso
  cantidad: number
}

const CLAVE = 'tienda-carrito'
const guardado = localStorage.getItem(CLAVE)
const carrito = ref<Linea[]>(guardado ? JSON.parse(guardado) : [])

function agregar(curso: Curso) {
  const linea = carrito.value.find((l) => l.curso.id === curso.id)
  if (linea) linea.cantidad++
  else carrito.value.push({ curso, cantidad: 1 })
}
function vaciar() {
  carrito.value = []
}

const cupon = ref('')
const mensaje = ref('')
const descuento = ref(0)

const cantidadTotal = computed(() => carrito.value.reduce((suma, l) => suma + l.cantidad, 0))
const subtotal = computed(() =>
  carrito.value.reduce((suma, l) => suma + l.curso.precio * l.cantidad, 0),
)
const igv = computed(() => subtotal.value * 0.18)
const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))

watch(
  carrito,
  (nuevo) => {
    localStorage.setItem(CLAVE, JSON.stringify(nuevo))
  },
  { deep: true },
)

watch(cupon, (nuevo, anterior) => {
  console.log(`cupón: "${anterior}" → "${nuevo}"`)
  mensaje.value = 'Validando…'
  setTimeout(() => {
    const valido = nuevo === 'VUE2026'
    descuento.value = valido ? 0.1 : 0
    mensaje.value = valido ? '✅ 10% de descuento' : '❌ Cupón no válido'
  }, 800)
})

watchEffect(() => {
  document.title = `Tienda (${cantidadTotal.value})`
})

const cargando = ref(true)
const cursos = ref<Curso[]>([])

setTimeout(() => {
  cursos.value = CURSOS
  cargando.value = false
}, 1500)

const busqueda = ref('')
const orden = ref<'titulo' | 'precio'>('titulo')
```

```
const visibles = computed(() => {
  const texto = busqueda.value.toLowerCase()
  return cursos.value
    .filter((c) => c.titulo.toLowerCase().includes(texto))
    .sort((a, b) =>
      orden.value === 'precio' ? a.precio - b.precio : a.titulo.localeCompare(b.titulo),
    )
})

const abierto = ref<number | null>(null)

function alternar(id: number) {
  abierto.value = abierto.value === id ? null : id
}

const cliente = reactive({ nombre: '', email: '' })
const { nombre } = toRefs(cliente)

const vFocus: Directive<HTMLElement> = {
  mounted: (el) => el.focus(),
}
</script>

<template>
  <h1>Tienda de cursos</h1>
  <p>Pedido de {{ nombre }} · {{ cantidadTotal }} cursos</p>

  <!-- Alumno -->
  <section>
    <h2>Alumno</h2>
    <input v-focus v-model.trim="cliente.nombre" placeholder="Nombre" />
    <input v-model="cliente.email" placeholder="Email" />
  </section>

  <!-- Catálogo -->
  <section>
    <h2>Catálogo</h2>
    <input v-model="busqueda" placeholder="Buscar curso…" />
    <select v-model="orden">
      <option value="titulo">Por título</option>
      <option value="precio">Por precio</option>
    </select>

    <p v-if="cargando">Cargando cursos…</p>
    <p v-else-if="visibles.length === 0">No hay cursos con "{{ busqueda }}".</p>
    <template v-else>
      <p>{{ visibles.length }} cursos</p>
      <ul>
        <li v-for="curso in visibles" :key="curso.id">
          <button @click="alternar(curso.id)">{{ curso.titulo }}</button>
          <button @click="agregar(curso)">Agregar</button>
          <p v-show="abierto === curso.id">{{ curso.categoria }} · S/ {{ curso.precio }}</p>
        </li>
      </ul>
    </template>
  </section>

  <!-- Carrito -->
  <section>
    <h2>Carrito <button @click="vaciar">Vaciar</button></h2>
    <ul>
      <li v-for="linea in carrito" :key="linea.curso.id">
```

```
        <input v-model.number="linea.cantidad" size="2" />
        {{ linea.curso.titulo }} × {{ linea.cantidad }}
      </li>
    </ul>
    <input v-model.lazy="cupon" placeholder="Cupón de descuento" />
    <p>{{ mensaje }}</p>
    <p>Subtotal: S/ {{ subtotal.toFixed(2) }}</p>
    <p>IGV (18%): S/ {{ igv.toFixed(2) }}</p>
    <p><strong>Total: S/ {{ total.toFixed(2) }}</strong></p>
  </section>
</template>
```

BLOQUE 3 · VUE 3.5 Y 3.6 10 MIN

Vue 3.5: la versión que tienes instalada

CONCEPTO props desestructurados

QUÉ ES

Desde Vue 3.5 sacas las props a variables y **siguen siendo reactivas**.

PARA QUÉ SE USA

Componentes que reciben datos del padre, con valores por defecto de JavaScript.

Paso 3 de 4

lo que escribes	→	lo que genera el compilador
`const { cantidad = 1 } = defineProps()`		`const props = defineProps()`
`precio * cantidad`		`props.precio * props.cantidad`
`watch(cantidad, …)`		`≈ watch(2, …) ×`

un número suelto no avisa de nada: error de compilación

Paso 4 de 4

lo que escribes	→	lo que genera el compilador
`const { cantidad = 1 } = defineProps()`		`const props = defineProps()`
`precio * cantidad`		`props.precio * props.cantidad`
`watch(() => cantidad, …)`		`watch(() => props.cantidad, …) ✓`

la función pregunta el valor cada vez que Vue la llama

Cómo funciona, paso a paso

1. `const { cantidad = 1 } = defineProps()` parece una copia… y una copia perdería la reactividad.
2. Pero el compilador reescribe cada uso: `cantidad` pasa a ser `props.cantidad`. Sigue conectado.
3. `watch(cantidad, …)` recibiría un número suelto: Vue no puede vigilar un 2. Error de compilación.
4. La solución: un **getter**, una función que pregunta cada vez: `watch(() => cantidad, …)`.

REGLA

Para vigilar una prop desestructurada, pásale una **función** a `watch`.

1 · Props desestructurados reactivos: antes y ahora

```
// Antes de 3.5
const props = withDefaults(
  defineProps<{ precio: number; cantidad?:
   number }>(),
  { cantidad: 1 },
)
props.cantidad
```

```
// Vue 3.5+
const { precio, cantidad = 1 } =
   defineProps<{ precio: number; cantidad?:
   number }>()
cantidad
```

2 · useTemplateRef

```
// <input ref="buscador" v-model="busqueda" />
const buscador = useTemplateRef('buscador')

function vaciar() {
  carrito.value = []
  buscador.value?.focus()
}
```

3 · onWatcherCleanup

Antes de volver a ejecutar un `watch`, Vue cancela el trabajo pendiente de la vez anterior: como cancelar un pedido cuando cambias de opinión.

Tienda.vue · el watch del cupón, con limpieza

```
import { watch, onWatcherCleanup } from 'vue'

watch(cupon, (nuevo) => {
  mensaje.value = 'Validando…'
  const timer = setTimeout(() => {
    descuento.value = nuevo === 'VUE2026' ? 0.1 : 0
  }, 800)

  // Si el cupón cambia antes de los 800 ms, se cancela la validación vieja
  onWatcherCleanup(() => clearTimeout(timer))
})
```

Ejemplo 6 de 6 · ItemCarrito con props reactivos

· 7 min

La línea del carrito pasa a ser su propio componente.

1. **Archivo nuevo** — `src/components/ItemCarrito.vue` con `const { titulo, precio, cantidad = 1 } = defineProps<…>()`.
2. **Tienda · Script** — importa `ItemCarrito`. **Template · Carrito** — reemplaza el texto de cada línea por `<ItemCarrito>`; el input de cantidad se queda.
3. **ItemCarrito · Script** — `watch(cantidad, …)` da un error del compilador; `watch(() => cantidad, …)` lo arregla.
4. **Tienda** — `useTemplateRef('buscador')`: al vaciar el carrito, el foco vuelve al buscador.

El código completo

src/components/Tienda.vue

```
<script setup lang="ts">
import {
  ref,
  reactive,
  toRefs,
  computed,
  watch,
  watchEffect,
  useTemplateRef,
  type Directive,
} from 'vue'
import { CURSOS, type Curso } from '@/cursos'
import ItemCarrito from './ItemCarrito.vue'

interface Linea {
  curso: Curso
  cantidad: number
}

const CLAVE = 'tienda-carrito'
const guardado = localStorage.getItem(CLAVE)
const carrito = ref<Linea[]>(guardado ? JSON.parse(guardado) : [])

function agregar(curso: Curso) {
  const linea = carrito.value.find((l) => l.curso.id === curso.id)
  if (linea) linea.cantidad++
  else carrito.value.push({ curso, cantidad: 1 })
}
const buscador = useTemplateRef('buscador')

function vaciar() {
  carrito.value = []
  buscador.value?.focus()
}

const cupon = ref('')
const mensaje = ref('')
const descuento = ref(0)

const cantidadTotal = computed(() => carrito.value.reduce((suma, l) => suma + l.cantidad, 0))
const subtotal = computed(() =>
  carrito.value.reduce((suma, l) => suma + l.curso.precio * l.cantidad, 0),
)
const igv = computed(() => subtotal.value * 0.18)
const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))

watch(
  carrito,
  (nuevo) => {
    localStorage.setItem(CLAVE, JSON.stringify(nuevo))
  },
  { deep: true },
)

watch(cupon, (nuevo, anterior) => {
  console.log(`cupón: "${anterior}" → "${nuevo}"`)
```

```
  mensaje.value = 'Validando…'
  setTimeout(() => {
    const valido = nuevo === 'VUE2026'
    descuento.value = valido ? 0.1 : 0
    mensaje.value = valido ? '✅ 10% de descuento' : '❌ Cupón no válido'
  }, 800)
})

watchEffect(() => {
  document.title = `Tienda (${cantidadTotal.value})`
})

const cargando = ref(true)
const cursos = ref<Curso[]>([])

setTimeout(() => {
  cursos.value = CURSOS
  cargando.value = false
}, 1500)

const busqueda = ref('')
const orden = ref<'titulo' | 'precio'>('titulo')

const visibles = computed(() => {
  const texto = busqueda.value.toLowerCase()
  return cursos.value
    .filter((c) => c.titulo.toLowerCase().includes(texto))
    .sort((a, b) =>
      orden.value === 'precio' ? a.precio - b.precio : a.titulo.localeCompare(b.titulo),
    )
})

const abierto = ref<number | null>(null)

function alternar(id: number) {
  abierto.value = abierto.value === id ? null : id
}

const cliente = reactive({ nombre: '', email: '' })
const { nombre } = toRefs(cliente)

const vFocus: Directive<HTMLElement> = {
  mounted: (el) => el.focus(),
}
</script>

<template>
  <h1>Tienda de cursos</h1>
  <p>Pedido de {{ nombre }} · {{ cantidadTotal }} cursos</p>

  <!-- Alumno -->
  <section>
    <h2>Alumno</h2>
    <input v-focus v-model.trim="cliente.nombre" placeholder="Nombre" />
    <input v-model="cliente.email" placeholder="Email" />
  </section>

  <!-- Catálogo -->
  <section>
    <h2>Catálogo</h2>
    <input ref="buscador" v-model="busqueda" placeholder="Buscar curso…" />
    <select v-model="orden">
      <option value="titulo">Por título</option>
```

```
      <option value="precio">Por precio</option>
    </select>

    <p v-if="cargando">Cargando cursos…</p>
    <p v-else-if="visibles.length === 0">No hay cursos con "{{ busqueda }}".</p>
    <template v-else>
      <p>{{ visibles.length }} cursos</p>
      <ul>
        <li v-for="curso in visibles" :key="curso.id">
          <button @click="alternar(curso.id)">{{ curso.titulo }}</button>
          <button @click="agregar(curso)">Agregar</button>
          <p v-show="abierto === curso.id">{{ curso.categoria }} · S/ {{ curso.precio }}</p>
        </li>
      </ul>
    </template>
  </section>

  <!-- Carrito -->
  <section>
    <h2>Carrito <button @click="vaciar">Vaciar</button></h2>
    <ul>
      <li v-for="linea in carrito" :key="linea.curso.id">
        <input v-model.number="linea.cantidad" size="2" />
        <ItemCarrito
          :titulo="linea.curso.titulo"
          :precio="linea.curso.precio"
          :cantidad="linea.cantidad"
        />
      </li>
    </ul>
    <input v-model.lazy="cupon" placeholder="Cupón de descuento" />
    <p>{{ mensaje }}</p>
    <p>Subtotal: S/ {{ subtotal.toFixed(2) }}</p>
    <p>IGV (18%): S/ {{ igv.toFixed(2) }}</p>
    <p><strong>Total: S/ {{ total.toFixed(2) }}</strong></p>
  </section>
</template>
```

src/components/ItemCarrito.vue

```vue
<script setup lang="ts">
import { watch } from 'vue'

const {
  titulo,
  precio,
  cantidad = 1,
} = defineProps<{
  titulo: string
  precio: number
  cantidad?: number
}>()

watch(
  () => cantidad,
  (nuevo, anterior) => {
    console.log(`${titulo}: ${anterior} → ${nuevo}`)
  },
)
</script>

<template>
  <span>{{ titulo }} × {{ cantidad }} = S/ {{ (precio * cantidad).toFixed(2) }}</span>
</template>
```

BLOQUE 3 · VUE 3.6 1 MIN

Vue 3.6: el mismo código, un motor nuevo

Al momento de esta sesión, Vue 3.6 está en **release candidate** (RC): una versión casi final, en prueba pública, que todavía no se usa en producción. La estable es la 3.5. **Tu Tienda funcionaría igual.**

alien-signals

El sistema de reactividad, reescrito por dentro sobre *alien-signals*: menos memoria y más velocidad, con la misma API.

Vapor Mode

Un modo de compilación que actualiza **directo** el elemento que cambió, sin pasar por el Virtual DOM. Es opcional, por componente, y solo con `<script setup>`.

Seis decisiones que vas a tomar en cada componente

NECESITO…	USO	EN LA TIENDA
guardar un valor	`ref`	`carrito`, `cupon`
guardar un objeto que va junto	`reactive`	`cliente`
un valor calculado a partir de otros	`computed`	`total`, `visibles`
hacer algo hacia afuera	`watch` · `watchEffect`	localStorage, título
mostrar u ocultar	`v-if` · `v-show`	catálogo, detalle
una lista filtrada u ordenada	`v-for` sobre un `computed`	catálogo, carrito

Para practicar

Agrega un botón **"Quitar"** en cada línea del carrito. Bonus: que el botón esté en `ItemCarrito` y avise a la Tienda con un `emit`.

Glosario

TÉRMINO	EN UNA FRASE
Reactividad	Cuando cambia un dato, todo lo que lo usa se actualiza solo.
Dependencia	Un dato que un cálculo o el template leyó; Vue lo anota para saber qué actualizar.
Proxy	El intermediario que devuelve `reactive`: vigila lecturas y escrituras.
Valor derivado	Un dato que se calcula a partir de otros.
Caché	El `computed` recuerda su resultado y solo recalcula si cambió algo de lo que depende.
Efecto secundario	Algo que el código hace hacia afuera: guardar, llamar a una API, cambiar el título.
`deep`	Vigilar lo que hay dentro de la caja, no solo si cambiaron la caja.
Modificador	Ajuste que se pega a una directiva con un punto.
Getter	`() => valor`: una función que pregunta el valor cada vez.
RC	*Release candidate*: versión casi final, en prueba.

Para repasar

Fundamentos de reactividad	`vuejs.org`
Computed	`vuejs.org`
Watchers	`vuejs.org`
Renderizado de listas	`vuejs.org`
Directivas personalizadas	`vuejs.org`
Anuncio de Vue 3.5	`blog.vuejs.org`